

Análise de cenários de execução de projetos de P&D

SIOD · Entrega da Atividade 02 · Rafael S. Valadão

1. Definição da entrada

A entrada primária é um diretório com arquivos .mat produzidos pelas simulações usando parâmetros gerados via LHS. Cada arquivo contém P, com a estrutura e os resultados temporais do projeto, e parametros, com uma linha para o cenário. São lidos cinco parâmetros: retrabalho esperado, trabalho não previsto esperado, pressão de cronograma, pressão gerencial e variação da capacidade.

Em P.Runtime, SimData contém 61 instantes, de 0 a 60 tu, para o projeto agregado e para suas sete atividades. Entre as séries disponíveis estão SimTime e PercAccomplished.

O conjunto completo possui 10.000 simulações e ocupa aproximadamente 1,53 GB, por isso não faz parte do repositório. Como entrada intermediária reproduzível, o repositório contém data/simulacoes_100/dataset_projeto.npz e metadata.json, derivados de 100 cenários sintéticos. O NPZ contém X com dimensão (100, 61, 6) e Y com dimensão (100, 61, 19).

2. Definição da transformação

- Localiza e ordena os arquivos .mat do diretório informado.
- Lê P e parametros em Python e valida a estrutura das simulações.
- Extrai os cinco parâmetros e as séries temporais do projeto agregado.
- Monta X com parâmetros e SimTime, e Y com 19 séries na ordem documentada.
- Calcula os indicadores de conclusão e grava dataset_projeto.npz e metadata.json.
- Valida NPZ e metadado, seleciona PercAccomplished como alvo e preserva os seis campos de entrada.
- Usa simulation_id existente ou atribui IDs posicionais de 0 a N-1.
- Aplica split por cenário: 70% treino, 15% validação e o restante para teste.
- Calcula média e desvio somente no treino e normaliza todos os conjuntos com esses parâmetros.
- Grava o dataset normalizado, o metadado de preparação e o resumo por cenário.

O fluxo é único; o modo define apenas o ponto de início: simulations começa nos .mat, generate-dataset encerra após o dataset intermediário, e dataset começa no NPZ já criado. A divisão usa internamente o valor fixo 42 para produzir sempre o mesmo split.

3. Implementação

A implementação está em scripts/pipeline.py e scripts/gerar_dataset.py. Toda a conversão está em Python e versionada no repositório. O código do simulador e os arquivos de simulação não são necessários quando a execução começa no dataset publicado.

- dataset DIRETORIO: inicia em dataset_projeto.npz; sem diretório, usa data/simulacoes_100/.
- simulations DIRETORIO: executa o fluxo completo a partir dos arquivos .mat.
- generate-dataset DIRETORIO: gera somente dataset_projeto.npz e metadata.json.

O parâmetro opcional --output-dir altera a pasta de saída. --max-simulations limita a leitura dos .mat e é destinado principalmente à validação com uma amostra pequena. Os exemplos completos estão em docs/execucao.md.

Os logs informam o modo, os diretórios, a quantidade de arquivos, o progresso da leitura, a estrutura encontrada, o split, a normalização, as validações e a gravação.

4. Validações

Grupo	Verificação	Resultado
Diretório	Existência da pasta e mínimo de sete simulações	Aprovado
Simulação	Presença de P e parâmetros em cada .mat	Aprovado
Projeto	Sete tarefas e oito registros em Runtime	Aprovado
Tabelas	Nomes, tipos numéricos, linhas e ordem das séries	Aprovado
Dataset	NPZ e metadata.json obrigatoriamente na mesma pasta	Aprovado
Estrutura	X e Y tridimensionais e compatíveis	Aprovado
Identificadores	Tipo inteiro, quantidade correta e sem repetição	Aprovado
Valores	Ausência de NaN e infinito nas entradas e no alvo	Aprovado
Cenários	Cinco parâmetros constantes nos 61 instantes	Aprovado
Tempo	SimTime crescente, comum e igual a 0-60 em 61 instantes	Aprovado
Domínios	Parâmetros nas faixas e progresso entre 0 e 100, sem queda	Aprovado
Split	Cobertura completa e ausência de sobreposição	Aprovado
Normalização	Parâmetros calculados somente no treino e valores finitos	Aprovado
Saídas	Reabertura dos três arquivos e conferência de dimensões	Aprovado

Qualquer falha interrompe a execução com uma mensagem ERROR. No fluxo completo, são produzidos:

- dataset_esn.npz: X e Y normalizados, IDs e índices de treino, validação e teste.
- metadata_esn.json: campos, dimensões, regra de split e parâmetros de normalização e inversão.
- resumo_cenarios.csv: parâmetros, conjunto, progresso final, conclusão, tempo e déficit por cenário.

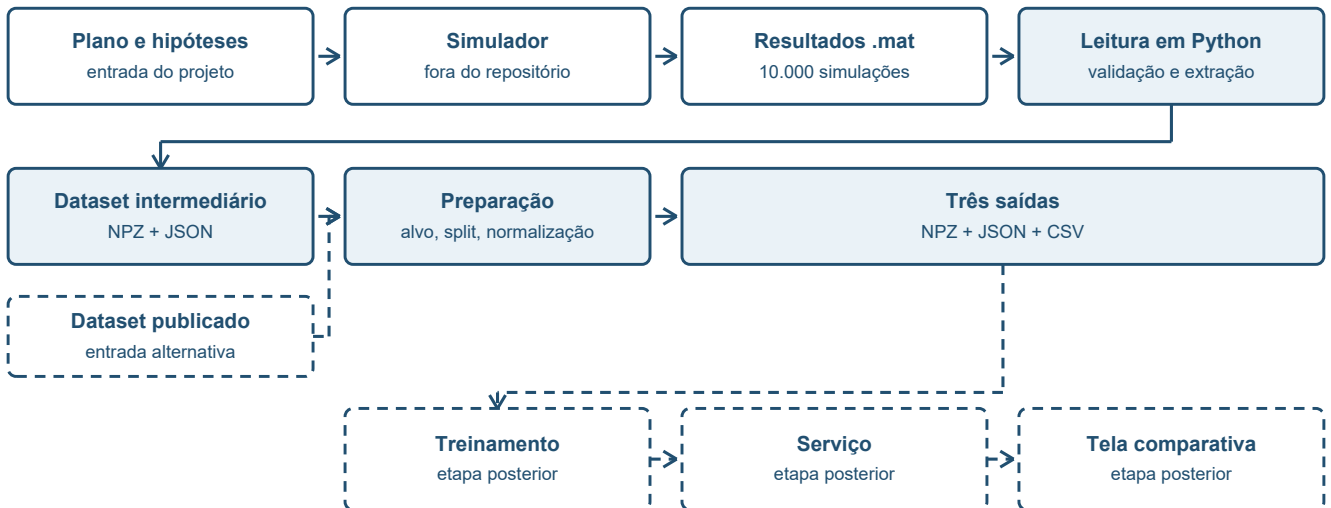
5. Evidência de execução

A execução padrão recebeu 100 cenários com 61 instantes. A saída contém X com dimensão (100, 61, 6), Y com dimensão (100, 61, 1) e 100 linhas no resumo. O split possui 70 cenários de treino, 15 de validação e 15 de teste. Foram observados 85 cenários concluídos e 15 não concluídos até 60 tu. Nenhum registro foi descartado, imputado ou corrigido.

As médias normalizadas do treino ficaram próximas de zero e os desvios próximos de um. Os conjuntos não se sobrepõem e cobrem os 100 cenários. A desnormalização recuperou os valores originais dentro da precisão float32, e o resumo reproduziu os indicadores calculados diretamente de PercAccomplished.

O fluxo Python também foi executado sobre dez arquivos .mat: foram identificadas sete tarefas, 61 instantes e 53 séries por simulação; o split foi 7/1/2. O modo completo e a geração isolada do dataset concluíram com sucesso.

6. Atualização da arquitetura



A arquitetura macro da Aula 01 foi mantida: plano e hipóteses alimentam o simulador, os resultados passam por validação e preparação, e uma base local sustenta as etapas posteriores de análise, serviço de consulta e tela comparativa.

A Atividade 02 detalha e implementa especificamente o trecho entre os resultados simulados e essa base local. Passaram a ficar explícitos a leitura dos .mat em Python, o ponto alternativo de entrada pelo dataset já criado, o split por cenário, a normalização sem vazamento e as três saídas. A base local deixou de ser representada apenas como um CSV genérico e passou a distinguir o NPZ de treinamento, os metadados e o resumo legível. O treinamento de uma rede neural, o serviço e a interface continuam como etapas posteriores.

7. Registro de decisão

A implementação revelou que os arquivos .mat podem ser convertidos em Python, sem depender do código privado do simulador, e que o resultado reproduz exatamente o dataset anterior. Também mostrou que o NPZ existente já contém as sequências necessárias para o treinamento de uma rede neural, mas ainda exige split por cenário e normalização calculada somente com o treino. O volume de aproximadamente 1,53 GB das simulações tornou necessário manter um ponto de entrada intermediário pelo dataset publicado. Por fim, ficou clara a necessidade de separar a entrada técnica de treinamento, seus metadados e um resumo legível para conferência e análise dos cenários.

Repositório: github.com/RafaelSV/siod-cenarios-projetos